

muchísimocks: An emulated cosmological simulation suite for simulation-based inference

Kate Storey-Fisher, Marcos Pellejero-Ibanez, Raúl Angulo, & friends [ksf: order?](#)

July 15, 2026

Galaxy clustering contains rich cosmological information down to non-linear scales; however, methods to extract this information are currently limited by the high cost of cosmological N -body simulations, as well as by the fidelity of galaxy bias modeling. We present a new approach to generating approximate but robust galaxy mocks: we employ the `map2map` full-field emulator to generate high-fidelity displacement fields, and apply hybrid Lagrangian bias expansion approach to flexibly model biased tracers. With this approach, we construct a suite of mock 3D galaxy density fields each of size $(1\ h^{-1}\ \text{Gpc})^3$ at $z = 0$ spanning 10,000 different cosmologies, called **muchísimocks**. To demonstrate the power of this data set, we use it to perform simulation-based inference (SBI) of galaxy clustering. We train an SBI model on summary statistics of the density fields—the galaxy–galaxy power spectrum, the galaxy–matter power spectrum, and the galaxy bispectrum—and infer the cosmological parameters, marginalizing over the bias and noise coefficients. We validate the pipeline on in-distribution test mocks and demonstrate the information content in the summary statistics, finding that the galaxy–matter power spectrum is critical for breaking parameter degeneracies, while the bispectrum does not add significant cosmological information. With this large suite, we are able to test convergence with the number of simulations, finding that while 1,000 cosmologies are sufficient for the power spectrum, upwards of 6,000 are required when including additional statistics. We also show that we recover unbiased cosmological constraints from an out-of-distribution model, an independent high-resolution N -body simulation populated with galaxies using extended SubHalo Abundance Matching, demonstrating the generalizability of the approach. This paves the way for exploring full-field inference from galaxy clustering, as well as a range of other potential applications. The **muchísimocks** suite and the code used in this paper are available at [thisurl](#).

1 Introduction

2 The muchisimocks suite

2.1 Emulating the displacement field with `map2map`

The nonlinear matter distribution can be simulated to high accuracy with N -body simulations. However, simulations with the resolution and volume required to analyze current galaxy surveys are very expensive, and training simulation-based inference models requires a very large of simulations. We thus turn to a field-level emulator of the matter distribution, `map2map` ?.

The density field can be approximated at linear order by the Zel’dovich approximation (ZA, also known as first-order Lagrangian perturbation theory, 1LPT), based on a realization of an initial conditions field. The `map2map` emulator learns a mapping from the ZA field at $z = 0$ to the non-linear N -body displacement field at that redshift, effectively learning the correction from linear to non-linear displacements. The emulator is trained on 2000 simulations of the Quijote simulation suite that span a latin hypercube (LH) of five cosmological parameters: Ω_c , σ_8 , h , Ω_b , and n_s (?). Each has a volume of $(1\ h^{-1}\text{Gpc})^3$ with 512^3 particles, giving particle mass resolutions in the range $2 \times 10^{11} - 1 \times 10^{12} h^{-1} M_\odot$. `map2map` also trains a separate network to predict the nonlinear velocity field from the ZA velocities. Once trained, `map2map` can generate the displacement and velocity fields for any input ZA field, taking a few minutes on a GPU for a $(1\ h^{-1}\text{Gpc})^3$ volume. While we do not utilize the velocity fields in this work, as we limit to real-space analysis, these will be important for future analyses including redshift-space distortions as well as other applications involving the velocity distribution; we release them as part of the `muchisimocks` suite. [ksf: explain special velocity field weights used, for zspace](#)

In more detail, `map2map` consists of two convolutional neural networks (CNNs), one to predict the matter density field and the other to predict the velocity field. The particle displacements are structured on a 3D grid as they are functions of their initial Lagrangian space position, so the CNN processes the inputs and outputs as 3-channel images. As most of the cosmological parameters only affect the input power spectrum that is used to create the ZA field, these are not needed as additional inputs to the model. However, the matter density Ω_m affects the N-body evolution, so is included as an input “style” parameter; this is incorporated via the approach of the StyleGAN2 (?), in which the style parameters modulate the convolution kernels. The model uses a V-net-like architecture, with 3 layers of downsampling and upsampling, and leaky ReLU activations. For its loss function, it combines the log mean squared error of the Lagrangian particle displacements and Eulerian density field.

For our mock suite, we define a latin hypercube of 10,000 sets of cosmological parameters: Ω_c , σ_8 , h , Ω_b , and n_s . We use uniform priors for each parameter that span ~ 10 around the ? values of the cosmological parameters, following ? (with a range for σ_8 extended to slightly lower values); these are shown in Table 1. For each of these, we generate a ZA field based on a different

random seed, and pass that field through `map2map` to obtain the nonlinear particle displacements. The `map2map`-emulated particle displacements are a key ingredient in modeling the galaxy bias fields, as discussed in the next section.

2.2 Modeling tracers with the hybrid bias expansion

To model biased tracers of the matter field, we use the hybrid Lagrangian bias expansion, also known as hybrid effective field theory (HEFT, ???). We first define a mapping from initial (Lagrangian) space coordinates $\mathbf{q}$ to Eulerian space coordinates $\mathbf{x}$ as $\mathbf{x} = \mathbf{q} + \Psi(\mathbf{q})$, where $\Psi(\mathbf{q})$ is the Lagrangian space displacement field. This advection from Lagrangian to Eulerian space effectively expresses the overdensity field as weighted contributions from the initial density field that end up in a given Eulerian location. The displacement term $\Psi(\mathbf{q})$ can be computed with Lagrangian perturbation theory, but this is only accurate down to mildly non-linear scales; here we take the “hybrid” approach which uses N -body displacements to compute $\Psi(\mathbf{q})$, significantly improving the accuracy down to non-linear scales.

The galaxy overdensity field in Lagrangian space can be described to second order as

$$\delta_g(\mathbf{q}) = 1 + b_1 \delta_L(\mathbf{q}) + b_2 [\delta_L^2(\mathbf{q}) - \langle \delta_L^2 \rangle] + b_{s^2} [s^2(\mathbf{q}) - \langle s^2 \rangle] + b_\Delta \nabla^2 \delta_L(\mathbf{q}) \quad (1)$$

where $\delta_L(\mathbf{q})$ is the linear matter density in Lagrangian space, and $s^2(\mathbf{q})$ is the shear field, defined by the traceless part of the tidal field, given by $s^2(\mathbf{q}) = s_{ij}(\mathbf{q}) s^{ij}(\mathbf{q}) = \delta_i \delta_j \phi(\mathbf{q}) - \frac{1}{3} \delta_{ij}^K \delta_L(\mathbf{q})$, where $\phi(\mathbf{q})$ is the linear gravitational potential in Lagrangian space $\nabla^2 \phi(\mathbf{q}) = 4\pi G \bar{\rho} \delta_L(\mathbf{q})$, and δ^K is the Kronecker delta. The coefficients b_1 , b_2 , b_{s^2} , and b_Δ are the Lagrangian bias parameters. The prior ranges of the bias parameters used in our SBI analysis are shown in Table 1, though we note that a tracer field could be constructed for any choice of bias parameters with our saved bias fields.

We build this field starting from a random realization of the initial Gaussian field on a $N_{\text{grid}} = 512^3$ grid in a $L = (1h^{-1} \text{Gpc})^3$ box, matching the resolution and size of the Quijote simulations that `map2map` was trained on, and use the ZA to compute approximate displacements. We pass the ZA displacement and velocity fields as input to `map2map` (§2.1), which outputs emulated N -body displacement and velocity fields. We compute the Lagrangian operators based on the linear density field, smoothing these with a damping scale of $k = 0.75h \text{Mpc}^{-1}$. We then perform the advection to Eulerian space: The tracers are deposited onto a grid in Eulerian space with a cloud-in-cell (CIC) assignment, with positions given by the simulation displacement field and weights given by Lagrangian operator. This gives five bias fields in Eulerian space, $\mathcal{O}_i$: weighting the Lagrangian density field by the constant 1 gives the homogeneous field; by $\delta_L(\mathbf{q})$, the linear density field; by $\delta_L^2(\mathbf{q}) - \langle \delta^2 \rangle$, the quadratic density field; by $s^2(\mathbf{q}) - \langle s^2 \rangle$, the tidal field; and by $\Delta^2 \delta(\mathbf{q})$, the Laplacian. These fields can be weighted by the bias coefficients and summed to produce an overall tracer field δ_g .

We perform two postprocessing steps on these fields. [ksf: fill in](#) To reduce storage space, we reduce the size of these fields by cutting the high- k modes: we remove modes beyond $k_{\text{Nyquist}} = \pi N_{\text{grid,cut}}/L \approx 0.4$, where $N_{\text{grid,cut}} = 128$, our reduced grid size. We do this by Fourier transforming each bias field, nulling the modes higher than k_{Nyquist} , and inverse Fourier transforming back to real space. At k scales higher than this our hybrid bias model loses accuracy anyway and we don't consider them in our inference, so we do not give up accuracy with this k -cut. We perform the same reduction on the velocity fields, cutting each component of the velocity vectors in k -space. We finally perform a deconvolution of the bias fields, using a linear interpolation kernel at the original grid size N_{grid} to correct for the smoothing by the CIC kernel. (Technically the deconvolution should be performed before the k -cut, but the results are effectively equivalent and this ordering is much faster.) This deconvolution could be done at the summary statistics level, but we choose to deconvolve the fields for ease of use.

The final main suite, designed as a training set, contains 10,000 sets of tracer position fields (5 Lagrangian bias fields advected to Eulerian space) and velocity fields, each at a different cosmology; each is of size $(1h^{-1} \text{ Gpc})^3$ and resolution 128^3 . We also generate multiple smaller suites for testing, detailed in §???. We call these suites **muchísimocks** (a portmanteau of the words “muchísimos”, Spanish for “many”, and “mocks”).

2.3 Modeling noise at the field level

[ksf: technically this is a postprocessing step only needed for the SBI part, where should it go?](#)

We require a model for stochastic noise in order to model realistic discrete galaxy populations. This can be modeled at the level of each summary statistic, but our goal is to make this suite usable for a wide range of statistics as well as field-level analyses; we thus desire a field-level noise model. Further, as we are considering higher-order statistics, we need to include non-Gaussian noise contributions. We thus turn to the noise model of ?, an EFT approach described by nonlinear couplings of a single Gaussian field to the bias fields. In this model the galaxy density field can be written as

$$\delta_g(\mathbf{x}) = \sum_{m=0}^{\infty} \sum_{1, \mathcal{O}} b_{\mathcal{O}}^{\{m\}} [\epsilon_G(\mathbf{x})]^m \mathcal{O}(\mathbf{x}) \quad (2)$$

where m is the order of the expansion in the noise field, $\epsilon_G(\mathbf{x}) = \mathcal{N}(0, 1)$ is a unit Gaussian field, $\mathcal{O}$ are operators of the galaxy density field, and $\{b_{\mathbb{1}}^{\{m\}}, b_{\mathcal{O}}^{\{m\}}\}$ are free coefficients. In our case we limit to second order in the bias expansion, as described in §2.2; limiting to zeroth order in m reduces to the noiseless case, as in Equation 1. For our SBI analysis, we include terms through first order in the noise expansion, $m = 0, 1$. That gives us bias coefficients $\{b_{\mathcal{O}}^{\{0\}}\} = b_1, b_2, b_{s^2}, b_{\Delta}$, as in Equation 1 (we fix $b_{\mathbb{1}}^{\{0\}} = 1$), and noise coefficients $\{b_{\mathcal{O}}^{\{1\}}\} = A_{\text{homog}}^{\text{noise}}, A_{b_1}^{\text{noise}}, A_{b_2}^{\text{noise}}, A_{b_{s^2}}^{\text{noise}}, A_{b_{\Delta}}^{\text{noise}}$.

We note that limiting the noise model to $m = 1$ means that we exclude noise contributions to $P_{gm}(k)$, as at $m = 1$ the noise field ϵ_G has zero mean (whereas $P_{gg}(k)$ and $B_{ggg}(k_1, k_2, k_3)$ get factors of ϵ_G^2 which don't vanish). This means that the bias parameters may have to absorb partial noise contributions, though we show that we still achieve largely unbiased bias parameters; future work could extend this noise model to $m = 2$ or choose a well-motivated subset of noise parameters tailed to the statistics analyzed.

Operationally, we generate one unit Gaussian noise field ϵ_G per cosmology in our suite. We store these and use them to construct the noisy tracer field on-the-fly just before computing the summary statistics of the field. This means that we use the same noise realization when we select different bias and noise parameter sets for the same cosmology (detailed in §?? and §??), which introduces some undesirable non-independence; however, we expect the effects of this to be minimal, and the storage cost of one noise field per bias model is prohibitive. The prior ranges of the noise parameters used in our SBI analysis are shown in Table 1.

3 Simulation-based inference methodology

3.1 Neural posterior estimation

The main idea behind simulation-based inference is that we can infer a posterior distribution of our data over the parameters of interest using only a *simulator*, which produces data samples given a set of parameters. This is distinct from explicit likelihood inference, in which we are required to define a likelihood of our data with respect to the parameters. Neural density estimators (NDEs) present a flexible and robust way to model a target distribution. Here, we use neural posterior estimation, which uses an NDE to directly model the posterior distribution.

To do this, we aim to minimize the distance between the true posterior $p(\theta|x)$ and the NDE model of the posterior $q_\phi(\theta|x)$, where ϕ are the parameters of the neural network model. We use maximum likelihood estimation, which minimizes the Kullback-Leibler divergence between the distributions. In practice, we consider the posteriors multiplied by $p(x)$:

ksf: update to using align

$$E = \dots \tag{3}$$

$$= L \tag{4}$$

where in the second line, we have converted the integral to a sum as we are using discrete samples from our simulation; in the third line, we observe that we are minimizing with respect to the neural network parameters ϕ and the true posterior $p(\theta|x)$ is independent of ϕ so it drops out; and in the third line we use the fact that minimizing the positive is the same as maximizing the negative.

Normalizing flows are a popular choice for the NDE, thanks to their expressivity and flexibility. The key idea is to transform an initial simple distribution

into the target distribution through a chain of transformations. Specifically we use masked autoregressive flows (MAFs), which stack Masked Autoencoders for Distribution Estimation (MADEs) in order to model more complex distributions. This is defined as

$$q_\phi(\theta|x) = \dots . \quad (5)$$

The MAF samples an initial base variable z_0 from a Gaussian $\mathcal{N}$, and then performs the transformations $z_t = f_t(z_{t-1}, \theta)$, where f_t is neural network that takes as input the previous variable z_t as well as the parameters θ . The Jacobian (the determinant term) arises from the change of variables, and involves f^{-1} , thus requiring f to be invertible. The invertibility property of NFs both enables us to sample from the network and gives us an explicit density, which is key for performing SBI. [ksf: via tucci paper, check](#) The loss function derived from the KL divergence (Equation 4) is used to train the weights and biases of the network.

3.2 Training set

To train our simulation-based inference model, we use the main `muchisimocks` suite of bias fields to generate a set of tracer fields. Our prior on the cosmological parameters is set by the range of the suite, as described in 2.1. We need to define the range of the bias coefficients $\{b\}$ we use to sum the fields; for this, we use the results of [?](#), which define priors that capture a range of realistic galaxy populations modeled with the extended subhalo abundance matching technique [?](#). We choose to use uniform priors with slightly wider ranges than the relations found by [?](#), a more conservative choice; these prior ranges are shown in Table 1. For the noise coefficients $\{A^{\text{noise}}\}$ we choose wide uniform priors loosely based off of the corresponding (noiseless) bias coefficients.

While we are limited to the 10,000 cosmologies defined in the `muchisimocks` suite, we can combine these into tracer fields with any number of bias parameter sets we choose. At a given cosmology, we then have multiple tracer fields generated with distinct sets of bias and noise parameters. Though this means the final tracer fields are not fully independent of each other, this has been shown (e.g. in [?](#)) to improve inference by significantly augmenting the training data size and allowing the model to disentangle cosmology and bias parameters. Note that the summing of bias fields with the chosen bias coefficients is essentially instantaneous and can be done in postprocessing just before computing summary statistics, and thus requires no additional compute time or storage.

As we later want to test convergence of the inference with the number of bias parameter sets, we use a nested latin hypercube to generate the bias parameters. The top-level LH contains 320,000 points, resulting in 32 independent bias parameter sets for each of the 10,000 cosmologies. Nested subsets of this LH are also LHs, with 16, 8, 4, 2, and 1 bias parameter sets per cosmology. This was achieved with the [?](#) algorithm for nested latin hypercubes, which works by ... [ksf: TODO](#). A python implementation of this algorithm along with valida-

Table 1: Uniform prior ranges for the cosmological, bias, and noise parameters. The cosmological parameters are based on a ~ 10 range around the ? best-fit values, and the bias and noise parameters are loosely based on the hypervolume found by ? for a wide range of realistic galaxy populations.

Parameter	Lower bound	Upper bound
<i>Cosmological parameters</i>		
Ω_c	0.23	0.40
σ_8	0.65	0.90
h	0.6	0.8
Ω_b	0.04	0.06
n_s	0.92	1.01
<i>Bias parameters</i>		
b_1	-1.0	3.0
b_2	-2.0	2.0
b_{s^2}	-2.0	2.0
b_Δ	-10.0	10.0
<i>Noise parameters</i>		
$A_{\text{homog}}^{\text{noise}}$	-3.0	3.0
$A_{b_1}^{\text{noise}}$	-3.0	3.0
$A_{b_2}^{\text{noise}}$	-2.0	2.0
$A_{b_{s^2}}^{\text{noise}}$	-5.0	5.0
$A_{b_\Delta}^{\text{noise}}$	-10.0	10.0

tion tests are available in the `gethypercube`¹ package. [ksf: should i include an appendix detailing the nested-LH package? or just link to the github as im doing? or does it deserve it's own little software paper?](#) This is shown to achieve LHs at each level that have properties much closer to a true LH than either taking random subsets of a top-level LH, or stacking bottom-level LHs. The use of LH-distributed parameters has been shown to be important for accurate and robust training, so we expect that this nested strategy leads to robustness in our convergence tests, which are a key result of this work. We generate one nested LH for the noiseless case with the 4-parameter set of bias parameters; and a separate nested LH for the noisy case with 4 bias parameters and 5 noise parameters, for a total of 9 independently varied parameters.

To generate our final tracer fields, we multiply the `muchisimocks` bias fields and noise field by the selected set of corresponding bias and noise parameters as in Equation (??), and sum these weighted fields. We normalize these fields by the original `map2map` grid size (512^3) before computing their summary statistics.

3.3 Summary statistics

We perform our SBI analysis with our `muchisimocks` framework on three standard summary statistics: the galaxy–galaxy power spectrum $P_{gg}(k)$, the galaxy–matter power spectrum $P_{gm}(k)$, and the bispectrum $B_{ggg}(k_1, k_2, k_3)$.

The galaxy–galaxy auto power spectrum $P_{gg}(k)$ is defined by

$$\langle \delta_g(\mathbf{k}) \delta_g(\mathbf{k}') \rangle = (2\pi)^3 \delta_D(\mathbf{k} + \mathbf{k}') P_{gg}(k) , \quad (6)$$

where $k = |\mathbf{k}|$ is the Fourier space wavenumber. This is the two-point function of the galaxy field, the most common statistic for cosmological inference from galaxy samples that captures all the Gaussian information. We use 32 logarithmically spaced bins from $k = 0.01$ to $k = 0.4$; the maximum k value is set by the Nyquist frequency of our mocks. We use the BACCO code to compute $P_{gg}(k)$ for our mocks. We can compute the 15 cross-terms of the 5 bias fields and later sum their contributions based on the bias parameters to obtain the power spectrum of the total tracer field; in practice we do this for the noiseless case, but in the noisy case compute $P_{gg}(k)$ for the total tracer field as this would add additional terms.

The galaxy–matter cross power spectrum $P_{gm}(k)$ is defined by

$$\langle \delta_g(\mathbf{k}) \delta_m(\mathbf{k}') \rangle = (2\pi)^3 \delta_D(\mathbf{k} + \mathbf{k}') P_{gm}(k) , \quad (7)$$

where $\delta_m(\mathbf{k})$ is the matter density field in Fourier space. This represents the galaxy–galaxy lensing contribution to 3x2pt analyses, and is critical for constraining galaxy bias and breaking parameter degeneracies. We use the same binning as $P_{gg}(k)$, and again use the BACCO code to compute $P_{gm}(k)$ for our mocks; for our fiducial results, we only use up to $k_{\max} = 0.25$ for $P_{gm}(k)$.

The galaxy auto-bispectrum $B_{ggg}(k_1, k_2, k_3)$ is defined by

$$\langle \delta_g(\mathbf{k}_1) \delta_g(\mathbf{k}_2) \delta_g(\mathbf{k}_3) \rangle = (2\pi)^3 \delta_D(\mathbf{k}_1 + \mathbf{k}_2 + \mathbf{k}_3) B_{ggg}(k_1, k_2, k_3) . \quad (8)$$

¹<http://github.com/kstoreyf/gethypercube>

This is the Fourier space three-point function. We choose 12 linearly spaced bin edges from $k = 0.01$ to $k = 0.4$, yielding 203 triangles; for our fiducial results, we only use triangles with edges up to $k_{\max} = 0.25$, yielding 50 triangles. We use the PolyBin3D code (??) to compute $B_{ggg}(k_1, k_2, k_3)$ for our mocks.

3.4 Implementation and training

Before training, we reparameterize the cosmology and bias parameters to combinations that are better constrained by the summary statistics, and thus should result in improved training convergence ?. We multiply the bias and noise parameters by the relevant power of σ_8 , obtaining $\sigma_8 b_1$, $\sigma_8^2 b_2$, $\sigma_8^2 b_{s,2}$, $\sigma_8 b_\Delta$, $\sigma_8 A_{b_1}^{\text{noise}}$, $\sigma_8^2 A_{b_2}^{\text{noise}}$, $\sigma_8^2 A_{b_{s,2}}^{\text{noise}}$, and $\sigma_8 A_{b_\Delta}^{\text{noise}}$. When sampling from the posterior, we can divide out the appropriate power of the recovered *sig* to obtain estimates of the original bias and noise parameters. We also do not include the cosmological parameters h , Ω_b , and n_s in the SBI training, as we do not expect to constrain these parameters (and confirmed in tests); these are still varied in the training set and implicitly marginalized over.

We use the publicly available `sbi` ? package for the NPE and MAF implementations. Our data vector is comprised of the binned statistics discussed in §3.3, concatenated when using multiple statics. The statistics combinations we show are: $P_{gg}(k)$, $P_{gg}(k) + B_{ggg}(k_1, k_2, k_3)$, $P_{gg}(k) + P_{gm}(k)$, $P_{gg}(k) + B_{ggg}(k_1, k_2, k_3) + P_{gm}(k)$ (we exclude $B_{ggg}(k_1, k_2, k_3)$ and $P_{gm}(k)$ alone, as these do not contain much constraining power on their own and in practice are almost always considered in combination with $P_{gg}(k)$). We use a 90%/10% training/validation split of our training set. We use the ADAM optimizer, and stop training when the loss over the validation loss does not improve for 20 epochs; we save the network weights at the best validation loss. We perform a sweep over several hyperparameters, using 30 random samples from the following sets: the learning rate $\{10^{-2}, 3 \times 10^{-3}, 10^{-3}, 3 \times 10^{-4}, 10^{-4}, 3 \times 10^{-5}, 10^{-5}\}$, hidden features $\{32, 64, 128, 256\}$, training batch size $\{32, 64, 128, 256\}$, and number of transforms $\{4, 6, 8, 10\}$. The sweeps are performed separately for each statistic or combination of statistics, and for each choice of noiseless (bias parameters only) and noisy (bias and noise parameters); we always use the maximum size training set of 320,000 data vectors, and for the statistics use $k_{\max}^{P_{gg}} = 0.4$, $k_{\max}^{P_{gm}} = 0.25$, and $k_{\max}^B = 0.25$. The model which achieves the minimum best validation loss is selected; we use this best set of hyperparameters to train variations of the model, such as when varying the number of training points and the $k_{\max}$ values. The best hyperparameters for the fiducial $P_{gg}(k) + P_{gm}(k) + B_{ggg}(k_1, k_2, k_3)$ noisy case are: learning rate 10^{-4} , hidden feature vector of size 256, batch size 256, and 6 transforms. The set of best hyperparameters vary across the statistics, with the noisy case (which varies more parameters) generally preferring larger hidden feature vectors and batch sizes, and most learning rates around 10^{-4} .

Training on the maximum size training set of 320,000 data vectors takes a few hours on a single CPU for the fiducial model described above. To display the posteriors and compute covariances and other statistics from them, we draw

10,000 samples from the posterior; we confirm that the posteriors are generally converged with this number of samples.

3.5 Test sets

We generate several test sets to validate our `muchisimocks` suite and our SBI approach.

Our first test set is a *cosmic variance set*. For this we use `muchisimocks` pipeline to construct 1000 additional mocks each with a different random seeds, all at the same fixed cosmology; we choose this to be the same cosmology as the out-of-distribution SHAMe mock test described below. We also fix the bias parameters to those measured from that mock. For the noise parameters, we cannot measure these from the mock; to identify realistic noise parameters, we generate 1000 sets of noise parameters and compute the statistics of the cosmic variance mocks assigning one of these noise parameter sets to each. We compute the reduced χ^2 between the statistics for the cosmic variance mocks and the SHAMe mock, considering $P_{gg}(k)$ and $B_{ggg}(k_1, k_2, k_3)$ with the fiducial k_{max} choices (the noise model does not affect $P_{gm}(k)$ as discussed in §2.3 so we exclude it); we choose the set of noise parameters that gives the lowest χ^2 . For the recovery tests, we compute the mean statistic over these 1000 mocks, and perform SBI on that mean to reduce the randomness from cosmic variance; in Appendix ?? we show that this gives the same result as taking the mean of the posteriors, while being much faster.

Our second test set is our *coverage set*. This consists of 1000 mocks generated with the `muchisimocks` pipeline that span the same range of cosmology, bias, and noise parameters as in the training set. This tests the performance of our method across the prior space.

Our final test set is our *SHAMe mock*; we construct a realistic catalog of galaxies using the extended subhalo abundance matching (SHAMe) approach ? applied to a BACCO simulation ?. This is a stringent test as it uses both a different, significantly higher resolution underlying cosmological simulation and a completely different galaxy formation model than the `muchisimocks` data used to train the inference network, testing the flexibility of our model. In detail, we use the BACCO simulation [ksf: which one?](#) with a box size of $(1024 h^{-1} \text{Mpc})^3$ (slightly larger than the `muchisimocks` training data) and 3072^3 particles, which uses the fixed-and-paired technique (?), and has cosmological parameters $\Omega_c = 0.309$, $\sigma_8 = 0.816$, $h = 0.677$, $\Omega_b = 0.0486$, and $n_s = 0.967$. Halos are found using the ? halo finder, and we populate these halos with galaxies using the SHAMe model with the following parameter choices: XXX. We choose a fiducial number density of $2.2 \times 10^{-4} (h^{-1} \text{Mpc})^{-3}$, similar to that of DESI emission line galaxies (ELGs) and slightly lower than DESI luminous red galaxies (LRGs). To check our inference results, we measure the bias parameters of this sample using the field-level probabilistic bias approach of ?; we find $b_1 = 0.474$, $b_2 = 0.127$, and $b_{s^2} = -0.339$ (we do not expect b_Δ to be well-measured by this approach).

For each of the two simulations of opposite phases from the fixed-and-paired

pair, the SHAMe model gives a set of discrete galaxy tracers. We deposit these to a mesh using the Cloud-in-Cell mass assignment scheme, of size $(512)^3$. We then remove the high- k modes and deconvolve the field in the same way as the `muchisimocks` fields. We compute the statistics on each of the two fields with the same method used for the training data, and take the mean of these two statistics as the data vector for inference.

4 Results

4.1 In-distribution recovery tests

4.1.1 Cosmic variance test set results

We first test our SBI pipeline on our cosmic variance test set, at fixed values of the cosmology, bias, and noise parameters. We compute summary statistics on the test data in the same way as on the training data.

ksf: check this whole paragraph when have final results! For the cosmic variance test set, we compute the mean of all of the statistics computed on the 1000 individual realizations and use this as the data vector for inference. The posteriors are shown in Figure ?? for three key parameters, Ω_m , σ_8 , and b_1 ; we infer and marginalize over the other bias parameters and the noise parameters but do not show them here. (The full contour plot is shown in Appendix ??.) We find that we obtain tight, unbiased constraints on the cosmology and bias parameters when using the full combination of statistics ($P_{gg}(k) + P_{gm}(k) + B_{ggg}(k_1, k_2, k_3)$), validating the inference approach.

While $P_{gg}(k)$ on its own provides significant constraining power on Ω_m , it results in a weak constraint on b_1 and a weak, biased constraint on σ_8 . The weakness of these constraints is expected as a result of the perfect degeneracy between σ_8 and b_1 in the power spectrum. Including $B_{ggg}(k_1, k_2, k_3)$ significantly increases the precision on b_2 and b_{s^2} , but interestingly only slightly increases the precision on Ω_m ; further, it does not improve the bias on σ_8 . We see that using $P_{gg}(k) + P_{gm}(k)$ produces unbiased constraints on σ_8 and b_1 ; this makes sense as $P_{gm}(k)$ is able to break this degeneracy (as it only contains one factor of b_1 , $P_{gm}(k) = b_1 \sigma_8^2 \hat{P}(k)$, compared to $P_{gg}(k)$ which contains two, $P_{gg}(k) = (b_1 \sigma_8)^2 \hat{P}(k)$ making the dependence only on $b_1 \sigma_8$). This corroborates our hypothesis that the bias observed in σ_8 for $P_{gg}(k)$ and $B_{ggg}(k_1, k_2, k_3)$ is likely a result of projection effects caused by a lack of constraining power. The full statistics combination $P_{gg}(k) + P_{gm}(k) + B_{ggg}(k_1, k_2, k_3)$ preserves the unbiased constraints, and the bispectrum adds some improvement of constraining power compared to $P_{gg}(k) + P_{gm}(k)$: it improves Ω_m by XX, σ_8 by XX, and b_1 by XX, and notably the bias parameters b_2 by XX, b_{s^2} by XX, and b_Δ by XX.

We note that these results are for real space, and we would expect some differences in redshift-space, including the bispectrum capturing more cosmological information than in the real-space case. ksf: double check that this is true

4.1.2 Coverage test set results

We infer the parameters of the 1000 samples of our coverage test set, spanning the prior range of our training set; this allows us to test the dependence of the estimated parameters on the true parameter values. We show distributions of the results in Figure ???. The top panel shows the fractional difference between the predicted and true value of the parameter; we can see that for all statistics combinations, the distribution peaks at the true value, indicating generally unbiased parameter inference. The width of the distribution shows the precision on the recovered parameters; we can see that including more statistics improves the precision. Specifically, including $P_{gm}(k)$... and additionally adding $B_{ggg}(k_1, k_2, k_3)$... This is generally consistent with our findings for the cosmic variance test set, though this averages over the parameter dependence of the recovery. The bottom panel shows the difference between the predicted and true parameter value divided by the error on the posterior, giving the number of sigmas away from the truth. If our inference is properly calibrated, this distribution should follow a unit Gaussian (e.g. 68% of the time, the recovered values should be within $1\sigma_8$ of the truth); indeed, we see that this is the case for all parameter combinations. [ksf: check](#)

In Figure ??, we show the predicted values of the parameters as a function of the true values. For $P_{gg}(k)$, we can see that the error bars on the parameters of interest are quite large, and tend toward the mean; this is expected when the statistic does not provide significant constraining power on the parameter, and results in a bias for parameters toward the edges of the prior bounds. When we include $P_{gm}(k)$, we can see that we greatly improve the constraints on σ_8 , breaking the degeneracy with b_1 as we saw in the cosmic variance test set. ... more analysis ...

These results show that our inference pipeline can accurately recover the cosmological and bias parameters for an in-distribution test set across a range of parameter values, and allow us to understand the information content in the statistics.

4.2 Dependence on number of cosmologies and bias parameter sets

We test the convergence of our inference with the number of training data points. For varying the number of cosmologies, we train the SBI network on a subset of our cosmologies. To vary the number of bias parameters, we populate each of those cosmologies with a varying number of bias parameter sets. We compute the mean squared error (MSE) over all the parameters for the coverage set samples; these errors as a function of the number of training points is shown in Figure ??.

We see that ...

4.3 Scale dependence

4.4 Application to SHAMe galaxy sample

The results of our SBI pipeline on our SHAMe out-of-distribution test set are shown in Figure ??, for the key parameters. We obtain generally unbiased constraints on the cosmological and bias parameters when using the full statistics combination. The trends generally follow those seen in the cosmic variance test set (§?? and Figure ??), with $P_{gm}(k)$ breaking the degeneracy between σ_8 and b_1 , and $B_{ggg}(k_1, k_2, k_3)$ improving constraints significantly for the bias parameters and mildly for the cosmological parameters. We do see biases in the values of b_1 and b_2 when using the full statistics combination; this could be a result of projection effects or of the bias parameters absorbing some model mismatch. [ksf: comment on whether the same thing is seen in CV set](#) However, the cosmological parameters remain unbiased. For the SHAMe data, we obtain a XXX% improvement in precision on Ω_m and XXX% on σ_8 when including $P_{gm}(k)$ compared to $P_{gg}(k)$ alone; the full statistics combination $P_{gg}(k) + P_{gm}(k) + B_{ggg}(k_1, k_2, k_3)$ results in XXX% and XXX% improvements on Ω_m and σ_8 respectively compared to $P_{gg}(k)$.

This test demonstrates the robustness of our forward model to modeling realistic galaxy populations constructed with completely different models.

5 Discussion

We also tested other SBI approaches, including Moment Networks. However, we found these to be...

There are many possible extensions to the model. We could extend the bias model to the third order, which has been shown to be important when... We could even generate larger volumes using `map2map`; volume is a main limitation of current SBI approaches.

There are many further potential uses of the `muchísimocks` suite. For instance, one could learn to reconstruct the velocity fields given the density fields. The suite could be used to explore the information in higher-order statistics of the density field, including optimal neural network compressions. One could also investigate the effect of more restrictive priors on the Lagrangian bias parameters, encoding our knowledge specific galaxy samples.

The smallest scales we can include are limited by... [ksf: when limited by map2map vs HBE?](#)

6 Summary and Conclusions

Acknowledgements

The authors would like to thank Jed (LMU), Beatriz Tucci, Matt Ho; Matteo Zennaro, Daniel López-Cano, Francisco Maion and the rest of the BACCO

group; and Risa Wechsler and the KIPAC Galaxy Formation and Cosmology group for helpful discussions.